Seed Evaluation for Concise iPRA

A local simplification with a complete proof checker

Prepared for Vladimir Reshetnikov.

Comparison pinned to ProveIt commit `443cff49563457a48ee0f663012d56c4bdb58fa0`.

Abstract

The current zero-normal scalar presentation of intuitionistic primitive recursive arithmetic uses an equality identifying a preceding iterate to evaluate the next iterate. This note replaces that premise by equality of the initial values:

$$a = b, \quad I(Sc, a; u.h) = r \qquad \Longrightarrow \qquad r = h\langle I(c, b; u.h)\rangle.$$

Zero-counter iteration remains a syntax convention. The replacement and the current rule derive one another using reflexivity and arithmetic inferences alone; no induction or logical rule is needed for that equivalence. The complete calculus still has five schemes. It retains the same numerical terms, formulas, scopes, and normalized substitution, so the whole-domain bi-interpretation with full first-order iPRA is preserved. The checker and proof-certificate comparisons are deliberately modest: the rule change ties the original implementation in one direct comparison and saves two normalized bytes and two tokens in a second matched implementation. Its standard computation certificates omit one repeated predecessor iterator. This is a verified local improvement, not a proof of absolute simplicity or a new compact-checker record.

1 Result, comparison, and exact limits

The requested target is full first-order intuitionistic primitive recursive arithmetic: all intuitionistic first-order formulas are allowed, while numerical induction is restricted to quantifier-free formulas. It is not enough to obtain the same standard numerical functions, the same equational theorems, or an equiconsistency statement without both interpretation round trips. The result here preserves the entire consequence relation of the native scalar clause calculus.

The pinned repository describes sixty-one research packages, a fully formalized native scalar bridge to iPRA, and several increasingly small arithmetic interfaces [1]. Its latest commit formalizes reflexivity plus the uniform successor-step rule in `NativeZeroStep.lean`. The five-scheme clause version is supplied in `Research/uniform_step`; the clause presentation itself is a written mathematical development, not one of that file's Lean theorems [2, 3].

Theorem 1 (Main local reduction)**.** *Let C_{Step} be the zero-normal five-scheme clause calculus in the pinned `uniform_step` package, and let C_{Seed} be the calculus explicitly specified below. For every well-scoped finite clause context Γ and formula A,*

$$C_{\mathrm{Step}} : \Gamma \vdash A \quad \Longleftrightarrow \quad C_{\mathrm{Seed}} : \Gamma \vdash A.$$

The end formulas and contexts are literally unchanged. Consequently the ordinary realization of C_{Seed} is equiconsistent and bi-interpretable with iPRA, with both numerical domains whole, equality actual, and both numerical composite isomorphisms given by $x = y$.

What is new relative to the inspected baseline. The new item is the fusion of initializer replacement with successor evaluation in one crossed rule. The earlier `initializer_equality` package has initializer replacement and a separate successor-computation axiom. The `counter_evaluation` package fuses evaluation with equality of counters in a different recursor language whose body binds both the old counter and the state [4, 5]. Here the iterator is the existing scalar iterator, its body binds one state variable, and the only arithmetic axiom is reflexivity. The case $c = 0$ supplies equality reasoning; the general case supplies computation. No literature-wide priority claim is made.

The size verdict. The deliverable includes the original checker, a matched shared-walker control, and the candidate. The candidate's smaller overall source comes mostly from sharing syntax validation and substitution traversal. The mathematical rule change contributes only the small residual saving in that implementation. The original helper-based implementations tie. Neither result beats the historical compact-checker frontier. Keeping these distinctions visible is necessary to assess whether the change is worth adopting under the repository's proof-checker-size metric.

Late repository update. While this work was in progress, the repository advanced to `56aafa39cf53a47dc4ae9f3a030fd8ac279a5624`, filing six more packages and formalizing the raw five-scheme clause calculus. Those additions were checked before delivery [10]. In particular, `clausal_single_evaluation` contains a 2,283-byte goal-directed Step checker; the same SeedEval adaptation also measures 2,283 bytes. Sharing the syntax traversal reduces both versions equally to 2,265 bytes. It already presents the short congruence/transport derivations used below, so those macros are not claimed as new. The exact initializer/evaluation fusion is absent from the inspected new presentations.

Assessment against the requested target. The construction meets the equiconsistency and bi-interpretability requirements and gives a concrete alternative arithmetic interface. It does *not* establish a convincing new minimum for complete proof-checker size. The positive results are the exact rule equivalence, the smaller standard computation certificates, and a very small local source saving. A more substantial simplicity improvement remains open.

Verification status. The new equivalence is proved in writing and supported by executable translations and certificate checks. The package does not claim a new Lean or Rocq verification. The inherited scalar-to-iPRA bridge is cited at its pinned repository source; its Lean build was not rerun in this environment. All new checker and certificate comparisons below are generated from the included files rather than inferred from instruction counts.

Contents

1 Result, comparison, and exact limits 1

2 Initializer evaluation: a smaller arithmetic interface 3

3 The full intuitionistic bridge 7

4 The standalone checker and its trust boundary 9

5 Measurements and reproducible evidence 11

6 A further structural reduction that did not win on size 13

7 Further research questions 14

Provenance and included materials 15

2 Initializer evaluation: a smaller arithmetic interface

The new rule compares an equality directly with an iterator's seed. The current Step rule instead compares it with the entire preceding iterate. Both rules give exactly the same arithmetic consequences, already without induction or any logical inference. On the same syntax and certificate protocol, the change also shortens the canonical certificate for each successor computation with a syntactically nonzero predecessor counter: its first premise becomes the reflexivity of the seed, rather than the reflexivity of a whole predecessor iteration.

This is a modest local simplification. It retains the same five-rule logical presentation, changes neither the numerical language nor its normalization, and does not claim a globally minimal proof checker. The separately reported whole-module measurements quantify its small implementation improvement; the proof below establishes the mathematical equivalence independently of those measurements.

2.1 Unchanged syntax and normalized substitution

Use raw scalar terms

$$t ::= x \mid 0 \mid St \mid I(n, a; u.h),$$

where u binds only its occurrences in h. Let N be the baseline's zero-only normalizer: normalize every child recursively, then replace an iterator whose counter is literally zero by its normalized initializer. Do not unfold a successor iteration. The numerical terms of the calculus are the fixed points of N.

For already normal arguments use the abbreviation

$$K(c, a; u.h) := \begin{cases} a, & c \text{ is literally } 0, \\ I(c, a; u.h), & \text{otherwise.} \end{cases}$$

The counter, initializer, and body have all already been normalized. Write $h\langle b\rangle := N(h[b/u])$, with capture-free substitution. All numerical substitutions in formulas and clauses have this same normalizing convention. The inherited coherence equations are

$$N(Nt) = Nt, \qquad N(t[\sigma]) = N\big(N(t)[N \circ \sigma]\big).$$

They follow by structural induction, lifting substitution under every binder. In particular, normalized substitution is associative and introduces no variable that was absent from the term and substitution images. No new normalization operation is added by the present proposal.

2.2 The arithmetic rule

Retain reflexivity and replace Step by

$$\boxed{\frac{\Gamma \vdash a = b \qquad \Gamma \vdash I(Sc, a; u.h) = r}{\Gamma \vdash r = h\langle K(c, b; u.h)\rangle}\ \text{SeedEval.}} \tag{SeedEval}$$

The same c, a, h occur in the displayed iterator. The seed equality does not change the counter, update body, or any passive parameter. In a checker, all three are read from the second premise. The first premise's left side is compared directly with a.

For comparison, the baseline has

$$\frac{\Gamma \vdash K(c, a; u.h) = t \qquad \Gamma \vdash I(Sc, a; u.h) = r}{\Gamma \vdash r = h\langle t\rangle}\ \text{Step.} \tag{Step}$$

Thus the new rule has not simply discarded a necessary equality check. Its conclusion is changed at the same time: it iterates from the equal seed b before performing the final update. Both rules coincide literally when $c = 0$:

$$a = b, \qquad I(1, a; u.h) = r \quad \vdash \quad r = h\langle b\rangle. \tag{ZStep}$$

The following elementary derivations use only this shared zero-counter instance and reflexivity.

2.3 A two-step equality pivot

For a normal term x, put $J_x := I(1, x; v.v)$. Given $p : x = y$ and $q : x = z$, apply ZStep to q and the reflexivity of J_x; then apply it to p and the result:

$$\dfrac{x = y \qquad \dfrac{x = z \qquad J_x = J_x}{J_x = z}\ \text{ZStep}}{z = y}\ \text{ZStep}. \tag{Pivot}$$

This works also when x is literally zero and under arbitrary retained assumptions. Taking one premise reflexive gives symmetry. Given $x = y$ and $y = z$, first reverse the former, then pivot at y; this gives transitivity. These facts require no induction, numerical cases, separation, or logical connective.

2.4 Congruence in two zero steps; crossed transport in four

Let $p : s = t$ and let $h(v)$ be any fixed normal numerical template, possibly with passive parameters and nested iterators. Set

$$H_s := I(1, s; v.h(v)), \qquad h_s := h\langle s\rangle, \qquad h_t := h\langle t\rangle.$$

First use ZStep on the reflexivities of s and H_s to obtain $H_s = h_s$. Reuse that result as the second premise of ZStep with first premise p. It gives

$$\dfrac{s = t \qquad \dfrac{s = s \qquad H_s = H_s}{H_s = h_s}\ \text{ZStep}}{h_s = h_t}\ \text{ZStep}. \tag{Cong}$$

The reuse of the first result as the minor premise is what makes congruence take two primitive steps. It is unnecessary to derive both $H_s = h_s$ and $H_s = h_t$ and pivot them.

Now suppose also $q : h_s = r$. Pivot the derived $h_s = h_t$ with q. The result is

$$s = t, \qquad h\langle s\rangle = r \quad \vdash \quad r = h\langle t\rangle. \tag{Tr}$$

It uses the two zero steps in (Cong) and the two zero steps in (Pivot): four primitive SeedEval instances in total. The argument is valid for every fixed numerical term template and every retained context. The same macro is valid in the old Step arithmetic, whose zero instances are identical. It is also present in the newly filed clause developments and is not claimed here as a new transport principle.

2.5 Every computation equation is derived

Zero computation is the definition of K. For successor computation, write

$$P := K(c, a; u.h), \qquad B := I(Sc, a; u.h).$$

Apply SeedEval to the reflexivities of a and B:

$$\frac{a = a \qquad B = B}{B = h\langle P\rangle}\ \text{SeedEval.} \tag{Comp}$$

This is exactly the baseline successor computation equation. Unlike the Step version, its major premise does not contain P.

2.6 SeedEval is derived from Step, without induction

Suppose the Step arithmetic has the two SeedEval premises

$$p : a = b, \qquad q : I(Sc, a; u.h) = r.$$

Use (Cong) in the fixed initializer template $v \mapsto K(c, v; u.h)$, with the hole variable v fresh for the counter and update body. It gives

$$K(c, a; u.h) = K(c, b; u.h).$$

Step with this equality as its first premise and q as its second premise gives exactly $r = h\langle K(c, b; u.h)\rangle$. Thus one SeedEval instance is simulated by two zero Steps and one generic Step. At $c = 0$, the original instance is already a zero Step and needs no expansion.

In ordinary arithmetic this is the same proof in familiar terms: equal seeds give equal iterates by congruence in the seed argument; successor computation then evaluates the displayed final step. No induction proving agreement of two different update bodies is used: the body h stays syntactically fixed throughout the argument.

2.7 Step is derived from SeedEval, without induction

Conversely suppose the SeedEval arithmetic has the two Step premises

$$p : P = t, \qquad q : B = r, \qquad P = K(c, a; u.h), \quad B = I(Sc, a; u.h).$$

The derived computation (Comp) gives $B = h\langle P\rangle$. Apply Pivot with q as its first premise and this computation as its second to obtain

$$h\langle P\rangle = r.$$

Apply the four-step crossed transport (Tr) in the body template h along $p : P = t$. Its result is $r = h\langle t\rangle$, the desired Step conclusion. The displayed expansion uses one computation SeedEval, two pivot applications, and four transport applications: seven primitive SeedEval instances. Again the case $c = 0$ is already a shared zero instance and can be translated directly.

It follows that the two arithmetic fragments derive exactly the same equations from every set of retained equations. More strongly, the translations can be inserted into arbitrary larger derivations while leaving every boundary formula unchanged. Each new inference is a finite derived inference of the other fragment. None of these equivalences invokes induction, case distinction, or any logical rule.

2.8 The complete logical and nonlogical specification

Use formulas and their ordinary readings

$$A ::= (s = t) \mid D_{x,y}B(x, y), \qquad (D_{x,y}B)^* := \exists x(B(x, 0)^* \to \forall y B(x, y)^*).$$

The designated contradiction is $0 = S0$. A retained clause is $[A; y.B]$, with reading $A^* \to \forall y B^*$; the displayed y binds only the body B. Write

$$\mathrm{Assm}(A) := [0 = 0; y.A] \qquad (y \notin \mathrm{FV}(A)).$$

The five primitive schemes are precisely:

$$\begin{aligned}
\text{Refl}: &\quad \Gamma \vdash t = t;\\
\text{SeedEval}: &\quad \frac{\Gamma \vdash a = b \qquad \Gamma \vdash I(Sc, a; u.h) = r}{\Gamma \vdash r = h\langle K(c, b; u.h)\rangle};\\
\text{Use}: &\quad \frac{[A; y.B] \in \Gamma \qquad \Gamma \vdash A}{\Gamma \vdash N(B[t/y])};\\
\text{DE}: &\quad \frac{\Gamma \vdash D_{x,y}B \qquad \Gamma, [N(B[b/x, 0/y]); y.N(B[b/x])] \vdash A}{\Gamma \vdash A};\\
\text{DI}: &\quad \frac{\Gamma, \mathrm{Assm}(N(B[t/x, \eta/y])) \vdash N(B[t/x, Sz/y])}{\Gamma \vdash D_{x,y}B}.
\end{aligned}$$

All displayed formulas and terms are normal and well scoped. In DE, b is fresh for the outer context, conclusion, and passive matrix parameters. In DI, z is fresh for the outer context, witness t, and passive parameters, and

$$\eta = \begin{cases} z, & B \text{ is a literal equation and } t \text{ is literally } 0,\\ 0, & \text{otherwise.} \end{cases}$$

Outer clauses are weakened when entering a fresh-variable scope. The chosen witness and the outer conclusion are checked in the outer scope before opening the branch. There are no further primitive equality, computation, successor, logical, or induction axioms. Use with the reflexivity of zero supplies the ordinary hypothesis rule for an encoded assumption.

2.9 Exact full-theory equivalence and bi-interpretation

Call the old five-scheme calculus C_{Step} and the new one C_{SeedEval}. They have literally the same terms, formulas, clauses, contexts, substitution, and freshness conditions. Replace each arithmetic inference in a derivation by its finite expansion above, and retain every logical inference unchanged. Structural induction on derivations gives, for every finite clause context Γ and formula A,

$$C_{\mathrm{Step}} : \Gamma \vdash A \quad \Longleftrightarrow \quad C_{\mathrm{SeedEval}} : \Gamma \vdash A. \tag{Exact}$$

The end formulas are identical, not merely provably equivalent. No witness is re-encoded or normalized differently, so the DI mode is unchanged. No new term-normalization theorem or new body-congruence argument is required for this passage between the two calculi.

The baseline's ordinary scalar realization is therefore also the ordinary realization of the new calculus. Its inherited bi-interpretation with full first-order intuitionistic primitive recursive arithmetic composes with the identity presentation map in (Exact). Both numerical domains remain whole, numerical equality remains actual equality, and the same provable identity graphs $x = y$ witness both composite isomorphisms. Formula round trips and all quantified consequences are inherited, since every sequent of the same native language is preserved. The contradiction $0 = S0$ is unchanged, so equiconsistency follows as well.

The new mathematical result is an explicit equivalence of arithmetic inference interfaces. The already established scalar-to-iPRA bridge, zero-normal syntax, clause logic, and witness-directed introduction are used with attribution; they are not claimed as new constructions.

2.10 An exact certificate-size improvement

Consider the standard tree certificate for one successor computation, with target $B = h\langle P\rangle$ as above. Its Step version has the proof tree

$$\mathrm{Step}(\mathrm{Refl}(P), \mathrm{Refl}(B)),$$

whereas SeedEval uses

$$\mathrm{SeedEval}(\mathrm{Refl}(a), \mathrm{Refl}(B)).$$

The number of proof-constructor nodes is unchanged. Count one node for a numerical constant or variable and one for each constructor. If $c \neq 0$ syntactically, then

$$|P| = 1 + |c| + |a| + |h|, \qquad |P| - |a| = 1 + |c| + |h|.$$

That is the exact reduction in supplied numerical syntax nodes in this proof tree. When $c = 0$, $P = a$ and the saving is zero. The public goal is identical in both certificates, so its size cancels in the comparison.

For the report's tagged compact JSON terms $I(c, a; u.h) = [\texttt{"i"}, c, a, h]$, with no optional whitespace, write $\ell(t)$ for the serialized term's UTF-8 length. For nonzero c, replacing the reflexivity of P by that of a saves exactly

$$8 + \ell(c) + \ell(h)$$

bytes in the proof field: the iterator's list/tag/separators contribute eight bytes, and the counter and body are no longer repeated there. The saving is unbounded as the body grows, although it is asserted only for these standard computation certificates.

This is not a claim that every SeedEval proof is shorter than every Step proof of the same result. General proof translations can expand derivations, retained clauses can provide short references, and a DAG certificate format may share subterms. The precise established advantage is removal of the repeated predecessor-iterator syntax from the canonical two-reflexivity computation proof, together with the separately measured complete-checker reduction.

3 The full intuitionistic bridge

3.1 Why a change in the arithmetic rule is enough

The preceding exact-sequent theorem is stronger than a standard-model soundness check. A proof may have arbitrary retained clauses, use a quantified formula as an assumption, discharge an assumption, or introduce a fresh numerical parameter. Replacing one arithmetic inference by its finite expansion leaves all its boundary formulas unchanged. It therefore fits inside every one of those proof contexts. The induction restriction and every eigenvariable condition remain the original ones.

The bridge being used has three layers:

$$C_{\mathrm{Seed}} \;\equiv\; C_{\mathrm{Step}} \;\equiv\; T_{\mathrm{eq}} \;\simeq\; \mathsf{iPRA}.$$

Here the first equivalence is this note's new theorem. The second is the existing clause/native-logic translation, with the usual zero-normal translation. The last is the repository's whole-domain

bi-interpretation. T_{eq} uses ordinary intuitionistic equality logic, the scalar iterator computations, numerical coverage, separation of 0 from $S0$, and induction for literal equations under arbitrary retained assumptions. Those are theorems or interpretations of the native presentation, not additional primitive rules appended to the five-scheme calculus.

3.2 The logical interface is explicit

For orientation, the conditional closure of a native formula can be represented using the one public binder:

$$\widehat{Q}_y B(y) := D_{x,y} B(y), \qquad x \notin \mathrm{FV}(B).$$

Its ordinary reading is equivalent to $B(0) \to \forall y B(y)$, since the domain has the closed inhabitant 0. Given this closure and a proof of $B(0)$, DE opens the retained clause $[B(0); y.B(y)]$; Use at t proves $B(t)$. Thus there is no missing primitive universal-elimination rule.

For equation progress, suppose $\Gamma, \mathrm{Assm}(E(z)) \vdash E(Sz)$, where E is a literal equation and z is fresh. DI at dummy witness 0 derives $\widehat{Q}_y E(y)$. Combining this with a base proof and closure elimination gives equation induction. This explains exactly where the arithmetic induction strength resides. It is not induction for arbitrary quantified formulas.

The case form of closure introduction can use the normal closed term

$$q := I(1, 0; u.u).$$

It is not the literal 0, while a single derived computation proves $q = 0$. At the dummy witness q, DI selects case mode. The inherited case-recovery argument also handles an atomic matrix whose intended existential witness is 0; the derivation and its freshness details are given in the supplied `uniform_step_theory.tex` reference.

The other intuitionistic connectives have the existing native macros. Let $\mathrm{sel}(v, A, B)$ be the recursively constructed formula which is A at $v = 0$ and B at a successor, and let $\mathrm{gate}(v, A)$ be true at zero and equivalent to A at a successor. These formula constructions are obtained from the term selector $I(v, a; u.b)$ and extended through the native binders. With the existing parity term par, one may use:

$$\begin{aligned}
\bot &:= 0 = S0, \\
A \to B &:= \widehat{Q}_y \,\mathrm{sel}(y, A, B), \\
A \wedge B &:= \widehat{Q}_y \,\mathrm{gate}(y, \mathrm{sel}(\mathrm{par}(y), A, B)), \\
\exists x\, A &:= D_{x,y} \,\mathrm{gate}(y, A(x)), \\
A \vee B &:= \exists v \,\mathrm{sel}(v, A, B), \\
\forall x\, A &:= A(0) \wedge \widehat{Q}_y A(y).
\end{aligned}$$

Their introduction and elimination rules and both logical round trips are part of the pinned native-logic bridge [1, 6]. The selected written foundational sources are included with this archive. The Seed equivalence preserves those derivations exactly, so it does not need to assume a new truth predicate, a function-variable sort, or classical excluded middle.

3.3 The two numerical interpretations

In the interpretation into iPRA, 0, S, and equality have their usual meanings. For each fixed finite body $h(u, \vec{p})$, the expression $I(n, a; u.h)$ denotes the primitive recursive function obtained by iterating that body n times from a, with $\vec{p}$ held fixed. The symbol describes one finite primitive recursive definition. There is no internal universal evaluator for arbitrary primitive recursive programs.

In the interpretation from iPRA, each primitive recursive function description is compiled into the already constructed scalar arithmetic. The existing bridge first constructs predecessor, then a faithful equality test, reflection of quantifier-free formulas, pairing, and a compiler for primitive recursion. These steps ensure the interpretation works on the entire internal domain, including nonstandard elements. They do not rely on agreement only at standard numerals. In particular, induction for equations transfers to induction for arbitrary quantifier-free formulas after that reflection theorem is available.

Composing either interpretation with the other returns the original numerical functions up to provable equality, with numerical identity map $x \mapsto x$. Formula round trips return provably equivalent formulas. The native and ordinary round-trip statements are present in the pinned `NativeBiInterpretation.lean` and `BiInterpretation.lean` sources [6, 7]. Since the new presentation map is the identity on all terms, formulas, and domains, it does not alter those maps or their proofs.

3.4 Equiconsistency means equivalence of contradiction proofs

A Seed proof of $0 = S0$ translates to an old native proof and then to an iPRA proof of falsity. Conversely, an iPRA proof of falsity translates through the inherited bridge and then to Seed. Hence

$$C_{\mathrm{Seed}} \vdash 0 = S0 \quad \Longleftrightarrow \quad \mathsf{iPRA} \vdash \bot.$$

This proves equiconsistency between the two presentations. It is not a claim that either theory proves its own consistency.

4 The standalone checker and its trust boundary

4.1 Concrete numerical and formula syntax

The main checker is a single standard-library Python module. It accepts a JSON object with exactly the fields `goal` and `proof`; the goal must be a closed, well-formed zero-normal formula. Its syntax is:

JSON object	Meaning
`0`	Numerical zero.
Positive integer j	The jth enclosing numerical binder, counted from one.
Negative integer $-j$	Rigid free-variable level j in the current scope.
`['s',t]`	Successor.
`['i',c,a,h]`	Iterator; only its body binds one variable.
`['=',s,t]`	Equation.
`['d',B]`	Paired binder; in B, index 1 is y and index 2 is x.

The printed examples use Python-like single quotes for readability; the actual certificates use JSON double quotes. A normal iterator never has literal counter zero. Opening a binder can expose such a counter, so opening recursively contracts the new zero iterators.

Each internal clause is a pair (A, B): A is in the current scope and B is under one additional bound variable. At a new eigenvariable scope, the fresh parameter receives the next unused negative level. Existing negative levels stay fixed. This representation implements the usual weakening of the old context without physically shifting all of its free-variable names.

4.2 Certificate constructors

The five instructions are direct encodings of the five schemes:

Certificate	Checked inference
`['r',t]`	Reflexivity of a validated normal term.
`['x',p,q]`	Seed, with both premise conclusions synthesized.
`['u',j,t,p]`	Use retained clause j, proving its antecedent with p, and instantiate its body at validated target t.
`['e',A,p,q]`	DE: p proves a paired formula, and q proves outer goal A under the fresh witness clause.
`['i',D,t,p]`	DI: D is a validated paired goal, t a validated witness, and p proves the prescribed successor instance under the appropriate assumption.

The goal and witness of a branch are validated in the outer scope before the new parameter is introduced. This prevents a witness or conclusion from depending on the supposedly fresh parameter. Clause indices are exact integers, not Boolean values or negative Python indices.

4.3 The two-line mathematical change

In the unchanged helper-based upstream checker, the old arithmetic tail is

```
require(s == iterator(n, a, h))
return ['=', r, opening(h, t)]
```

The candidate replaces it by

```
require(s == a)
return ['=', r, opening(h, iterator(n, t, h))]
```

This version has exactly the same measured normalized size as its control. It makes the mathematics particularly easy to audit: the first premise changes from a predecessor-iterate equality to a seed equality, and the missing predecessor iterator moves into the conclusion.

The main delivered implementation shares a single typed traversal between syntax validation and normalized opening. In that implementation the smart constructor is expressed inline, so the same rule change saves the pair of parentheses needed around a conditional expression in the old comparison. That is the source of the two-byte and two-token matched improvement. It is a small syntax-level consequence of a different valid rule, not evidence of a major decrease in the mathematical content.

4.4 Soundness and completeness of the acceptance interface

The proof of checker soundness is induction on a successful call to `derive`. Reflexivity validates its term. Use obtains a scoped antecedent/body pair from an already valid context. For Seed, the induction hypotheses ensure that both synthesized equations are valid and normal; pattern matching identifies the second equation's successor iterator, the explicit comparison identifies its seed, and normalized opening constructs exactly the displayed conclusion. DE and DI implement their fresh-variable premises in the enlarged scope and compare the resulting formula with the required branch target.

The same induction gives the normality and scope invariant for every returned formula. The traversal checks all children before contracting a zero iterator; an erased child cannot conceal a malformed term. Its use of exact integer and list types excludes Boolean indices and non-syntax objects. In the opener, inserted terms have only rigid free levels at the outer scope; locally bound indices remain protected by the depth counter. No arbitrary open bound-index tree is inserted without this invariant.

Conversely, encode any finite derivation of the five schemes recursively using the five displayed certificate constructors, choosing consecutive negative levels for its eigenvariables. Every rule check then succeeds. This supplies completeness for the specified finite syntax and proof objects. The public verifier restricts this general internal statement to closed theorems with empty context.

The trusted checker contains no `assert`, `eval`, or `exec`. Explicit exceptions reject. Standard Python resource limits still apply to a concrete run: stack or memory exhaustion is rejection or an unfinished computation, not an accepted proof. The mathematical acceptance relation is understood with unbounded finite resources; no claim is made that every derivation fits one process's default recursion limit.

4.5 What executable validation establishes

The included translator reconstructs a source proof's conclusion at each node and replaces arithmetic instructions by the derived macros proved above. Its output is checked again by the independently selected target checker. Tests cover numerical computation, equality symmetry, transitivity and transport, clause use, paired introduction and elimination, induction that actually uses its hypothesis, fixed passive parameters, and nested numerical binders. Negative certificates exercise bad indices, malformed syntax, invalid seed matches, wrong conclusions, and eigenvariable escape. Public-process tests also run with `python-O`, where Python assertion checks are disabled.

Finite tests can catch errors in the implementation and the translators; they do not replace the universal written equivalence proof. The exact counts and measurement environment appear in the generated evidence table below and in `evidence/validation.json`.

5 Measurements and reproducible evidence

5.1 Whole-module source measurements

All measurements use CPython 3.12.14. For comparability with the repository, the script removes a module's leading docstring, parses the complete module, and applies `ast.unparse` with a final newline. Normalized UTF-8 bytes count the resulting source; significant tokens exclude comments, encoding, indentation, newline, and end-marker tokens; AST nodes count `ast.walk`. JSON input handling, syntax checks, substitution, all inference rules, exceptions, and the public command-line entry point are included. No helper implementing trusted work is excluded.

Complete implementation	Bytes	Tokens	AST
Original computation/transport control	3,048	860	949
Pinned Step, original helper style	2,950	829	902
SeedEval, identical helper style	2,950	829	902
Step, shared forward traversal	2,804	758	815
SeedEval, shared forward traversal	2,802	756	815
Latest Step, goal-directed	2,283	741	826
SeedEval, matched goal-directed	2,283	741	826
Step, shared goal-directed traversal	2,265	709	787
SeedEval, shared goal-directed traversal	2,265	709	787

The strongest claim justified by the matched forward pair is a reduction from 2,804 to 2,802 bytes and from 758 to 756 tokens, with 815 AST nodes in both programs. The difference is only about 0.071% of the matched source bytes. The reduction from the shipped 2,950-byte checker to the 2,802-byte candidate is about 5.02%, but 146 of the 148 saved bytes are attributable to the shared traversal refactoring, which also benefits Step. The helper-based pair and the newer goal-directed pair both tie. These controls prevent assigning the whole implementation gain to the new mathematical rule.

The separate goal-directed traversal refactoring saves 18 bytes, 32 tokens, and 39 AST nodes in both versions. It is therefore a common implementation improvement, not a consequence of SeedEval. Gzip compression also does not favor the new rule: the direct goal-directed pair uses 818 versus 822 bytes, and the shared-traversal pair uses 872 versus 877 bytes (Step versus SeedEval, respectively). Those compressed figures illustrate why no single small source difference should be read as an invariant simplicity result.

The repository also contains a historical 516-byte compact codec and a 788-byte native public-goal checker [8, 9]. Their syntax and certificate-validation contracts differ from the tagged, strictly scoped forward modules here. They are useful evidence of how much source compression remains possible; neither lower byte count is beaten by this proposal. The separate best historical token, AST, and byte coordinates should not be treated as the measurements of one program. This report makes no claim of improving that compact lower envelope.

5.2 Certificate sizes

The standard direct computation family has the exact label saving proved above. The reproduction script also records complete compact-JSON byte counts, including the unchanged public goal. Representative examples are shown below. These are ordinary tree certificates; they do not assume external subterm sharing or an omitted term table.

Predecessor counter	Step bytes	SeedEval bytes	Saved
0	206	206	0
1	362	314	48
2	392	338	54

The smaller certificate uses the same three proof constructors: one evaluation node and two reflexivity leaves. The saving comes entirely from replacing the repeated predecessor iterator by its initializer. For increasingly large counters or bodies this saving grows without bound in absolute bytes. General translated proofs can become larger, as the seven-step simulation demonstrates, so these figures do not imply a global proof-length comparison.

5.3 Validation receipt

Check family	Count
Generated positive cases across interfaces	249
Pinned upstream positive certificates	83
Matched literal/shared comparisons	166
Checked proof translations	166
Checked proof round trips	166
Negative rejections	92
Independent opening/semantic comparisons	540
Independent numerical SeedEval checks	180
Open-sequent translation checks	4
Fixed-passive-parameter checks	3
Main public normal/optimized processes	398
Late goal-comparison positive pairs	8
Late goal-comparison negative cases	8
Late goal-comparison public processes	96

The package records the detailed counts, the exact Python version, source hashes, negative-case names, and translation-rule counts. The untrusted test harness may use ordinary assertions; the trusted acceptance modules use explicit checks. The tests are designed to resolve concrete risks: arithmetic source matching, zero contraction after substitution, binder capture, preservation of passive data, logical branch scoping, and accidental reliance on Python's assertion mode.

To rerun the delivered evidence from the package root:

```
python3 reproduce.py
python3 code/checker.py < code/examples/computation.json
python3 -O code/checker.py < code/examples/computation.json
```

The report's source is standalone. Two ordinary `pdflatex` runs rebuild its PDF. Regenerating the mathematical prose is not a prerequisite for rerunning the code checks or source measurements.

6 A further structural reduction that did not win on size

An independently useful alternative factors every successor iteration into an opaque one-step application. Introduce a suspended numerical application $U(a; u.h)$ with the intended reading $I(1, a; u.h)$, and normalize only by

$$I(0, a; u.h) \longrightarrow a, \qquad I(Sn, a; u.h) \longrightarrow U(I(n, a; u.h); u.h).$$

No reduction is performed on U. The arithmetic rule then becomes

$$\frac{a = b \qquad U(a; u.h) = r}{r = N(h[b/u])}.$$

It has no arbitrary counter. Every old Step becomes one application of this rule after factorization, and the same zero-step pivot recovers equality. The two unconditional rewrite rules are orthogonal. A direct positive-integer weight proves termination, including duplicated bodies:

$$\begin{aligned} w(0) = w(x) &= 1, & w(St) &= w(t) + 1, \\ w(U(a; u.h)) &= w(a) + w(h) + 1, \\ w(I(n, a; u.h)) &= (w(n) + 1)(w(a) + w(h) + 1). \end{aligned}$$

The successor factorization decreases this weight by exactly $w(a) > 0$; the zero rewrite decreases it as well. Every constructor is monotone in its child weights. Normalization is therefore total, confluent, and substitution coherent. It never executes an iterator body. The supplied notes give both presentation maps, both round trips, and the formulation that retains the old term grammar by using $I(1, a; u.h)$ as an abbreviation instead of a new constructor.

The complete implementation is the decisive qualification. In the shared forward format, the explicit-U checker measured 2,902 bytes, 803 tokens, and 864 AST nodes, against 2,804, 758, and 815 for its Step control. Its normalization and extra constructor cost more than the simplified arithmetic match saves. It is therefore included as a proved structural alternative, not as the recommended checker-size improvement. The standalone source, matched control, measurement script, and 83 checked positive certificates are supplied in `notes/unit_experiment/`.

This outcome also explains why moving computation into syntax is not automatically a simplification. The syntax constructor, its normalization, its scope handling, and its validation must all remain inside the cost boundary. Deleting an inference rule while omitting those costs would give the wrong comparison.

7 Further research questions

1. **A larger mathematical reduction.** Can the arithmetic interface be reduced beyond reflexivity and one evaluation scheme while retaining a small, explicitly counted syntax layer? The all-false interpretation of atomic equations obstructs simply deleting reflexivity from the unchanged clause calculus. Relocating it to an initial universal clause still leaves a reflexivity axiom to count.

2. **Proof sharing and initializer evaluation.** Can an initializer-oriented arithmetic kernel reduce certificate size for broader families of proofs, including nontrivial preceding-state equalities, without the seven-step translation overhead? The exact tree-certificate saving should also be compared against DAG certificates and explicit subterm tables, with those decoders counted.

3. **Partial normalization with a real net gain.** Is there a useful normalization between zero contraction and full numerical evaluation that reduces both the arithmetic interface and the complete trusted implementation? Suspended application gives a clean equivalence and a primitive recursive normalizer, but the measured implementation here does not meet that stronger target.

4. **Narrower induction principles.** The existing successor-rigidity approach derives induction for zero equations, but its faithful equality-reflection bootstrap remains a separate obligation. Can it be completed without importing general equation induction at an intermediate step? Counter-aware recursion gives a simpler predecessor, but does not by itself discharge the reflection argument. This note does not claim a solution.

5. **Formal verification and a common benchmark.** Add the SeedEval constructor and its equivalence to the existing `NativeZeroStep` development, then connect the same proof to the formalized clause layer. Separately, benchmark the compact, goal-directed, and forward checkers under explicit common contracts for syntax validity, open sequents, certificate sharing, and error behavior. A lower bound for one encoding should not be presented as an absolute mathematical lower bound for all presentations of iPRA.

Provenance and included materials

The primary comparison source is commit `443cff49563457a48ee0f663012d56c4bdb58fa0`; the late novelty and current-checker audit uses `56aafa39cf53a47dc4ae9f3a030fd8ac279a5624`. Selected original sources, their exact paths, and hashes are included under `reference/` and `evidence/`. The repository's MIT No Attribution license is included. New code, the written reduction, and the measurement scripts are supplied in source form.

The mathematical claims are narrower than an assertion that the requested absolute simplest axiomatization has been found. The archive supplies a fully specified, equivalent alternative and reproducible evidence of its benefits and costs, including the negative result of the stronger normalization experiment.

References

[1] ProveIt contributors. *Concise presentations of intuitionistic primitive recursive arithmetic*. README at commit `443cff495634`. Pinned repository source.

[2] ProveIt contributors. *NativeZeroStep.lean*, at the same commit. Definitions and equivalence theorems.

[3] ProveIt contributors. *Uniform successor evaluation on zero-normal terms*. `Research/uniform_step/source/theory.tex`, at the primary commit. Five-scheme specification and written bridge.

[4] ProveIt contributors. *A deterministic equality rule with no template parameter*. `Research/initializer_equality/sections/seed_rule.tex`, at the primary commit. Earlier initializer-replacement rule.

[5] ProveIt contributors. *Counter evaluation*. `Research/counter_evaluation/ipra_counter_evaluation.tex`, at the primary commit. Counter-aware comparison.

[6] ProveIt contributors. *NativeBiInterpretation.lean*, at the primary commit. Native/iPRA interpretation and round trips.

[7] ProveIt contributors. *BiInterpretation.lean*, at the primary commit. Ordinary scalar/iPRA bi-interpretation.

[8] ProveIt contributors. *Canonical recursors*, compact codec and its stated contract. Pinned compact checker files.

[9] ProveIt contributors. *Checked refinement*, native public-goal checker. Pinned source.

[10] ProveIt contributors. *ConciseIPRA README and newly filed packages*, commit `56aafa39cf53a47dc4ae9f3a030fd8ac279a5624`. Late comparison snapshot.